Pokémon Search Engine and Type Prediction

Docente: Ana Catarina Martins

Unidade Curricular: Text Mining

Trabalho Realizado por:

Guilherme Santos, Nº120090, gmfts@iscte-iul.pt

Bruno Bento, Nº120081, bcmbo@iscte-iul.pt

3.º ano, 2º Semestre – Ano Letivo 2024/2025

Resumo

Este projeto apresenta o desenvolvimento de duas aplicações de mineração de texto no contexto do universo Pokémon. A primeira aplicação, *Pokémon Searcher*, consiste num motor de busca semântico capaz de interpretar descrições subjetivas fornecidas pelo utilizador (ex: "forte e rápido") e devolver os Pokémons mais compatíveis, com base nos seus atributos estruturados. A segunda aplicação, *Type Predictor*, utiliza técnicas de Processamento de Linguagem Natural (PLN) e algoritmos de Machine Learning para prever automaticamente o tipo de um Pokémon (ex: fogo, água) a partir da sua descrição textual biológica, recolhida por Web Scraping da Bulbapedia. Foram testados três modelos de classificação — Regressão Logística, Random Forest e Naive Bayes — com a Regressão Logística a apresentar o melhor desempenho. O projeto demonstra a aplicabilidade de técnicas de Text Mining em domínios não convencionais, como o entretenimento digital, integrando dados estruturados e não estruturados num sistema útil e interativo.

Palavras-Chave: Text Mining, NLP, Classificação de Texto, Pokémon, TF-IDF, Logistic Regression, Random Forest, Naive Bayes, Web Scraping, Fuzzy Matching

Índice

Resumo ..... *i*

Índice ..... *ii*

Índice de Figuras ..... *ii*

Índice de Tabelas ..... *iii*

1. Introdução *1*

1.1 Contexto 1

1.2 Objetivos 2

2. Revisão de Literatura *2*

3. Metodologia *3*

3.1 Entendimento do Negócio 3

3.2 Entendimento dos Dados 3

3.3 Preparação dos dados 5

2.1.1. *Pokemon Seacher* 5

2.2.2. *Pokemon type predictor* 8

3.4 Modelação dos dados 13

2.2.1. *Random Forest* 13

2.2.2. *Regressão Logística* 14

2.2.3. *Naive Bayes* 14

3.5 Avaliação do Modelo 14

4 Conclusões *16*

5 Referências *18*

Índice de Figuras

Figura 1. Valores nulos: a) Antes; b) Depois. 5

Figura 2. Distribuição dos dados e estabelecimento de valores de referência. 6

Figura 3. Exemplo de um resultado que o utilizador pode obter. 8

Figura 4. Exemplos das descrições retiradas da Bulbapedia. 9

Figura 5. Distribuição dos tipos dos Pokemons após balanceamento. 10

Figura 6. Coluna desc_clean. 11

Figura 7. Tópicos NMF aplicados. 12

Figura 8. Os dez termos mais relevantes obtidos pela métrixa 13

Índice de Tabelas

Tabela 1. Valores obtidos para as métricas em relação aos modelos utilizados. 15

1. Introdução

Com a crescente disponibilidade de dados sobre personagens fictícias em jogos e plataformas online, torna-se cada vez mais viável aplicar técnicas de ciência de dados a domínios antes considerados exclusivamente de entretenimento. Neste contexto, o universo Pokémon representa um excelente caso de estudo, combinando dados estruturados (estatísticas de combate, características físicas) com descrições em linguagem natural (histórias e biologia dos Pokémons), permitindo explorar tanto análises quantitativas como processamento de linguagem natural (NLP).

O presente trabalho tem como objetivo desenvolver duas aplicações complementares baseadas em dados do mundo Pokémon. A primeira consiste num **sistema de pesquisa inteligente**, capaz de interpretar descrições subjetivas fornecidas pelo utilizador (como "um Pokémon rápido e forte") e devolver uma lista de Pokémons que melhor correspondem a essas características. A segunda foca-se na **classificação automática do tipo de um Pokémon** com base apenas na sua descrição biológica, utilizando modelos de aprendizagem supervisionada e técnicas de vetorização de texto.

Para isso, foram utilizados dados públicos provenientes do Kaggle e complementados com descrições mais detalhadas recolhidas por Web Scraping da plataforma Bulbapedia. O trabalho inclui todas as etapas fundamentais de um projeto de ciência de dados: desde o entendimento do problema e preparação dos dados, até à modelação e avaliação dos modelos de machine learning utilizados. O resultado final visa não só demonstrar a aplicabilidade de técnicas de NLP e classificação em contextos criativos, como também proporcionar uma ferramenta interativa e útil para fãs e curiosos do universo Pokémon.

1.1 Contexto

A profusão de descrições textuais em bases de dados online demanda ferramentas de mineração de texto para consulta e análise. No universo Pokémon, cada criatura possui descrições ricas que podem ser exploradas para buscas avançadas e previsão automática do seu tipo.

1.2 Objetivos

Desenvolver um índice de busca de Pokémons: Este objetivo consiste na criação de um sistema de pesquisa que permite ao utilizador descrever um Pokémon ideal utilizando linguagem natural, e receber como resposta os Pokémons mais compatíveis com essa descrição. Para isso, foi desenvolvido um mecanismo que interpreta os termos usados pelo utilizador e os associa aos atributos numéricos e tipos disponíveis no dataset original.

Construir e avaliar modelos de classificação de texto: Neste objetivo, o foco foi a construção de modelos de machine learning capazes de prever o **tipo de um Pokémon** (como fogo, água, etc.) com base apenas na sua descrição textual. Utilizando descrições obtidas por Web Scraping e processadas com técnicas de NLP, como limpeza, lematização e vetorização TF-IDF.

2. Revisão de Literatura

A classificação de texto é uma tarefa essencial no Processamento de Linguagem Natural (PLN), permitindo a categorização automática de grandes volumes de texto não estruturado. Técnicas como análise de sentimentos, deteção de spam e recomendação de conteúdos dependem diretamente da eficácia dos classificadores usados (Camacho-Collados & Pilehvar, 2018). O Naive Bayes, um modelo probabilístico baseado no Teorema de Bayes, é amplamente utilizado devido à sua simplicidade e rapidez. Embora a sua principal limitação seja a suposição de independência entre características — frequentemente violada em dados textuais — ele continua eficaz em tarefas como filtragem de spam e classificação de sentimentos (Manning, Raghavan & Schütze, 2008).

A Regressão Logística, por sua vez, é um modelo discriminativo que prevê a probabilidade de uma classe através de uma função logit. Oferece vantagens como interpretabilidade e saída probabilística, tornando-a útil para aplicações onde é importante entender a influência de cada variável (Jurafsky & Martin, 2021). No entanto, enfrenta limitações em cenários não-lineares e com variáveis correlacionadas, além de não capturar a ordem ou semântica das palavras quando usada com técnicas simples como Bag-of-Words (Zhang, Jin & Zhou, 2010).

O algoritmo Random Forest, por outro lado, é um método de ensemble que constrói múltiplas árvores de decisão baseadas em amostragem e subconjuntos aleatórios de características. É robusto ao overfitting, eficaz com dados ruidosos e de alta dimensionalidade, e apresenta resultados competitivos em termos de ac e F1-score (Breiman, 2001). Em estudos comparativos com o dataset 20 Newsgroups, Random Forest superou o Naive Bayes e a Regressão Logística em desempenho, embora com tempo de treino ligeiramente superior (Kowsari et al., 2019).

A escolha do classificador ideal depende de um equilíbrio entre acurácia, interpretabilidade e eficiência computacional. Enquanto o Naive Bayes é vantajoso em contextos com poucos recursos computacionais, a Regressão Logística fornece maior transparência e o Random Forest maximiza desempenho em cenários complexos. Assim, a seleção do modelo deve considerar os requisitos específicos da aplicação, bem como a forma de representação textual adotada — como TF-IDF ou embeddings contextuais (Minaee et al., 2021).

Apesar do avanço de modelos baseados em *deep learning*, os métodos tradicionais mantêm relevância prática. São menos exigentes em termos de dados e recursos, oferecem melhor interpretabilidade e ainda competem em performance, especialmente quando combinados com engenharia de características eficaz (Camacho-Collados & Pilehvar, 2018). A melhoria contínua das representações vetoriais de texto, como word embeddings e embeddings contextuais, pode amplificar ainda mais o potencial desses modelos clássicos.

3. Metodologia

3.1 Entendimento do Negócio

O projeto divide-se em duas componentes principais:

- **Pokemon Searcher**: Um sistema de recomendação baseado em texto livre, onde o utilizador descreve um Pokémon ideal (ex.: "forte, tipo fogo, pesado"), e o sistema devolve Pokémons do dataset que mais se aproximam dessa descrição. O objetivo é facilitar a busca por Pokémons com determinadas características físicas e estatísticas.
- **Pokemon Type Predictor**: Um modelo de classificação que, a partir da descrição biológica de um Pokémon, prevê o seu tipo. Esta funcionalidade é útil para análises automáticas, categorização de dados e exploração de padrões linguísticos associados a diferentes tipos de Pokémon.

3.2 Entendimento dos Dados

Para a realização deste projeto, foram utilizados três conjuntos de dados distintos, com origens e finalidades diferentes, que se complementam nas duas tarefas principais: a pesquisa de Pokémons com base em descrições do utilizador e a previsão do tipo do Pokémon a partir da sua descrição biológica.

O primeiro dataset utilizado, **pokemon.csv**, foi obtido da plataforma Kaggle e contém dados estruturados sobre 801 Pokémons. Este conjunto reúne atributos quantitativos como height_m, weight_kg, hp, attack, defense, speed e base_total, bem como informações categóricas como name, type1 e type2. Este ficheiro é essencial para a primeira parte do projeto — o sistema de pesquisa — pois permite fazer correspondência direta entre os termos qualitativos fornecidos pelo utilizador (como "forte", "rápido", "leve") e os valores numéricos desses atributos. De notar que nem todos os Pokémons possuem um segundo tipo (type2), por isso os valores nulos nesta coluna foram substituídos por “none”, assegurando consistência na filtragem. Este dataset foi também limpo de valores nulos noutras colunas, mantendo-se apenas os registos completos para análise.

O segundo dataset, **pokedex.csv**, também proveniente do Kaggle, tem uma estrutura mais simples, com colunas como name, type, speed, hp, attack e uma breve info textual sobre cada Pokémon. Embora útil como ponto de partida, constatou-se que a descrição (info) era demasiado curta e limitada para a tarefa de classificação textual, uma vez que muitas frases eram genéricas ou pouco informativas. No entanto, este ficheiro foi útil na preparação do Web Scraping, pois serviu de base para a construção das URLs de pesquisa para cada Pokémon no site Bulbapedia. Além disso, algumas colunas foram aproveitadas como apoio para verificação cruzada de tipos e atributos básicos.

O terceiro e mais importante ficheiro para a segunda parte do projeto foi o **merged_pokemon_biology.csv**, gerado a partir de Web Scraping à Bulbapedia. Este processo automatizado extraiu a secção de Biologia da página de cada Pokémon, recolhendo descrições mais detalhadas e ricas do ponto de vista semântico. O dataset final contém 1025 registos, com três colunas principais: name, description e type. Ao contrário dos outros dois, este conjunto tem como foco principal a análise de texto, já que cada Pokémon possui uma descrição em linguagem natural que serve como base para a tarefa de classificação. A coluna type foi utilizada como variável-alvo na modelação supervisionada.

Dado que a distribuição de tipos era bastante desigual (com alguns tipos a aparecerem muitas vezes e outros muito raramente), decidiu-se filtrar o conjunto para incluir apenas os sete tipos mais comuns: grass, fire, water, bug, normal, electric e psychic. Posteriormente, o dataset foi balanceado manualmente, através de amostragem estratificada, de forma a garantir que todos os tipos estivessem representados com o mesmo número de exemplos, evitando assim enviesamentos nos modelos de classificação.

3.3 Preparação dos dados

2.1.1. *Pokemon Seacher*

Na primeira fase do trabalho, procedeu-se à importação do ficheiro pokemon.csv, recorrendo à função pandas.read_csv(), tendo sido posteriormente selecionadas apenas as variáveis consideradas relevantes para a análise orientada ao utilizador, nomeadamente: name, type1, type2, height_m, hp, attack, defense, speed, weight_kg e base_total. Esta seleção teve como objetivo otimizar o desempenho computacional e centrar a análise nas dimensões mais informativas do ponto de vista do perfil e desempenho dos Pokémon.

De seguida, realizou-se o pré-processamento dos dados, com especial destaque na gestão de valores em falta (*missing values*). Neste contexto, realizou-se a eliminação de registos com valores nulos em variáveis críticas e, no caso particular da coluna type2, optou-se por substituir os valores nulos pelo rótulo *none*. Tal decisão fundamentou-se no facto de nem todos os Pokémons apresentavam um segundo tipo, sendo alguns do tipo monotipo e outros do tipo dual, e garante a uniformização dos dados categóricos, permitindo uma futura codificação eficiente e sem perdas de informação semântica. Estes parâmetros encontram-se visíveis na Figura 1:

```
Valores nulos antes:
name            0
type1           0
type2         384
height_m       20
hp              0
attack          0
defense         0
speed           0
weight_kg      20
base_total      0
dtype: int64
```

```
Valores nulos depois:
name          0
type1         0
type2         0
height_m      0
hp            0
attack        0
defense       0
speed         0
weight_kg     0
base_total    0
dtype: int64
```

Figura 1. Valores nulos: a) Antes; b) Depois.

De seguida, procedeu-se à análise estatística descritiva das variáveis numéricas, com o objetivo de caracterizar a distribuição dos dados e estabelecer valores de referência para atributos como *height_m*, *weight_kg*, *hp*, *attack*, *defense*, *speed* e *base_total*. Este passo é fundamental para, em fases subsequentes da pesquisa, validar a coerência entre os resultados obtidos por métodos de mineração de texto e os perfis numéricos efetivamente observados,

permitindo aferir a correspondência entre as descrições textuais e os dados quantitativos subjacentes. A sua representação encontra-se na **Figura 2**:

	height_m	hp	attack	defense	speed	weight_kg	base_total
count	781.000000	781.000000	781.000000	781.000000	781.000000	781.000000	781.000000
mean	1.163892	69.148528	77.693982	73.002561	66.279129	61.378105	428.774648
std	1.080326	26.621679	32.223734	30.806010	28.904200	109.354766	119.466916
min	0.100000	1.000000	5.000000	5.000000	5.000000	0.100000	180.000000
25%	0.600000	50.000000	55.000000	50.000000	45.000000	9.000000	323.000000
50%	1.000000	65.000000	75.000000	70.000000	65.000000	27.300000	440.000000
75%	1.500000	80.000000	100.000000	90.000000	85.000000	64.800000	508.000000
max	14.500000	255.000000	185.000000	230.000000	180.000000	999.900000	780.000000

Figura 2. Distribuição dos dados e estabelecimento de valores de referência.

Posteriormente, foram desenvolvidas e aplicadas três funções principais sobre a descrição textual fornecida pelo utilizador, com o objetivo de a preparar para análise semântica e correspondência com os dados estruturados do *dataset*:

1. **remove_acentos**: Esta função realiza a normalização ortográfica do texto, removendo acentuação gráfica e convertendo todos os caracteres para minúsculas. Para tal, recorre-se à biblioteca **unidecodedata**, o que assegura uniformidade na representação textual e previne falhas de correspondência causadas por variações diacríticas.

2. **Tokenização e criação de *n*-gramas**: Através da função **text.split()**, o texto é segmentado em unidades léxicas elementares (*tokens*). Posteriormente, são gerados *bigramas* por concatenação de *tokens* adjacentes, os quais são adicionados à lista de unidades lexicais para permitir uma correspondência mais robusta, especialmente no caso de termos compostos ou expressões frequentes.

3. **match_with_levenshtein**: Esta função implementa um mecanismo de correspondência aproximada entre *tokens* e termos de referência, com base na **distância de Levenshtein**. Este método permite identificar termos semanticamente próximos mesmo na presença de erros ortográficos ou variações mínimas de grafia, o que aumenta a tolerância e flexibilidade do sistema de pesquisa.

A função principal, **search_pokemon**, inicia-se com o pré-processamento da descrição introduzida pelo utilizador, incorporando as funções acima referidas. Neste ponto, é também definido um dicionário que contém os termos qualitativos e descritivos associados a cada atributo numérico (por exemplo, “alto” para *height_m* ou “ataque alto” para *attack*), bem como extraídos os tipos únicos presentes na base de dados (*type1* e *type2*), que servirão como referência para a correspondência com os *tokens* da descrição.

A correspondência entre os *tokens* da entrada e os termos de referência é realizada com o auxílio da métrica **rapidfuzz.fuzz.ratio**, que permite calcular a similaridade entre pares de termos. Os termos considerados suficientemente semelhantes são associados nas listas **matched_terms** (atributos descritivos) e **matched_types** (categorias de tipo Pokémon).

Para cada termo qualitativo detetado em **matched_terms**, é aplicado um filtro condicional ao *dataframe* original com base nos quartis das distribuições dos atributos numéricos correspondentes: valores superiores ao 3.º quartil (>75%) são classificados como "altos", enquanto valores inferiores ao 1.º quartil (<25%) são classificados como "baixos". Estes critérios permitem transformar descrições linguísticas subjetivas em operações quantitativas objetivas. Adicionalmente, caso existam correspondências em **matched_types**, o *dataframe* é filtrado de forma a incluir apenas os registos cujos tipos correspondam aos especificados pelo utilizador, refinando assim os resultados da pesquisa.

Além disso, foi criada a variável **score**, responsável por quantificar o grau de correspondência entre a descrição fornecida pelo utilizador e os atributos efetivos de cada Pokémon obtido. Esta métrica é expressa em percentagem e calculada com base na proporção de critérios descritos pelo utilizador que são efetivamente satisfeitos pelo registo em análise. Por exemplo, se a descrição contiver três atributos, como “tipo fogo”, “pesado” e “ataque alto”, e um Pokémon apresentar conformidade com os três, o **score** atribuído será de **100%**. Caso apenas dois dos três critérios sejam cumpridos, o **score** correspondente será de aproximadamente **66,7%**, o que reflete uma certa parcialidade da correspondência.

Este índice de semelhança desempenha um papel fundamental na ordenação dos resultados devolvidos, de modo a assegurar que os Pokémons mais alinhados com os requisitos expressos na descrição do utilizador são apresentados em primeiro lugar. A ordenação decrescente por **score** tem como objetivo maximizar a relevância semântica dos resultados, o que promove uma experiência de pesquisa mais intuitiva, realista e centrada no utilizador. Por fim, para evitar a sobrecarga cognitiva do mesmo, e garantir uma apresentação

concisa dos resultados, o sistema devolve apenas os cinco Pokémons com maior pontuação. Um resultado exemplo é demonstrado na **Figura 3**.

```
Descreva o Pokémon (ex: rápido, pesado, ataque alto, tipo fogo...): type fire pesado ataque alto
    name  type1  type2  height_m  hp  attack  defense  speed  weight_kg  base_total  score (%)
   Ho-Oh   fire flying       3.8 106     130       90     90      199.0         680        100
Reshiram dragon   fire       3.2 100     120      100     90      330.0         680        100
   Entei   fire   none       2.1 115     115       85    100      198.0         580        100
Camerupt   fire ground       1.9  70     120      100     20      220.0         560        100
Arcanine   fire   none       1.9  90     110       80     95      155.0         555        100
```

Figura 3. Exemplo de um resultado que o utilizador pode obter.

2.2.2. *Pokemon type predictor*

Na segunda parte do trabalho, começou-se por implementar um processo automatizado de recolha de descrições textuais dos Pokémons, com recurso a técnicas de **Web Scraping**, realizadas num ambiente Jupyter Notebook. Para este efeito, utilizou-se a biblioteca **requests** para efetuar as requisições HTTP às páginas da **Bulbapedia**, uma enciclopédia digital especializada em conteúdos do universo Pokémon, em conjunto com a biblioteca **BeautifulSoup** para o processamento e extração estruturada do conteúdo HTML.

O foco da extração incidiu unicamente na secção de *Biologia* de cada página individual do Pokémon, dado o seu interesse para efeitos de análise semântica. Para automatizar a construção dos URLs de acesso, recorreu-se a um *dataset* auxiliar (**pokedex.csv**), a partir do qual se iterou sobre os nomes dos Pokémons devidamente normalizados (isto é, convertidos para minúsculas, sem acentuação e com formatação compatível com os URLs da Bulbapedia). O conteúdo relevante foi extraído até ao cabeçalho **<h2>**, sendo recolhidos todos os parágrafos **<p>**, listas **<ul>** e itens **<li>** presentes até esse ponto, com o intuito de capturar uma descrição biológica representativa.

Durante o desenvolvimento, foi identificada uma limitação significativa no *dataset* original, onde as descrições textuais associadas a cada Pokémon eram excessivamente sucintas, ou seja, frequentemente reduzidas a frases simplificadas para efeitos de exibição em cartões ou aplicações, o que não proporcionava dados linguísticos com densidade e complexidade suficientes para uma análise conclusiva. Esta limitação motivou a adoção do processo de Web Scraping acima descrito.

Após a recolha dos textos, os dados extraídos foram integrados com os dados estruturados do *dataset* auxiliar através de uma operação de *merge*, resultando num *dataset* final enriquecido, contendo as colunas **name**, **description** e **type**. Este novo conjunto de dados foi guardado em formato .csv, com o objetivo de evitar a repetição do processo de extração e garantir a reprodutibilidade da análise subsequente. Algumas descrições encontram-se na **Figura 4**:

	name	description	type
0	bulbasaur	Bulbasaur is a small, quadrupedal amphibian Po...	grass
1	ivysaur	Ivysaur is a quadrupedal amphibian Pokémon tha...	grass
2	venusaur	Venusaur is a squat, quadrupedal amphibian Pok...	grass
3	charmander	Charmander is a bipedal, reptilian Pokémon wit...	fire
4	charmeleon	Charmeleon is a bipedal, reptilian Pokémon . I...	fire

Figura 4. Exemplos das descrições retiradas da *Bulbapedia*.

Com base no novo *dataset* criado, que contém um total de 1025 Pokémons, procedeu-se à sua leitura através da função **pandas.read_csv()**, o que permitiu uma exploração inicial das variáveis disponíveis. Uma análise preliminar revelou que os Pokémons estão distribuídos por 18 tipos distintos, nomeadamente: *normal*, *fire*, *water*, *electric*, *grass*, *ice*, *fighting*, *poison*, *ground*, *flying*, *psychic*, *bug*, *rock*, *ghost*, *dragon*, *dark*, *steel* e *fairy*. Esta elevada diversidade tipológica, combinada com uma distribuição assimétrica do número de Pokémons por tipo, motivou uma redução controlada do *dataset*.

Para mitigar o desequilíbrio entre classes e facilitar análises posteriores, optou-se por restringir a análise aos sete tipos mais frequentes. Esta seleção foi realizada através do seguinte comando:

"top_types = df_final['type'].value_counts().nlargest(7).index.tolist()"

Os tipos resultantes desta filtragem foram: grass, fire, water, bug, normal, electric e psychic, o que assegurou assim uma amostra estatisticamente representativa e operacionalmente mais eficiente.

De seguida, foi realizada uma operação de balanceamento do *dataset*, com o objetivo de prevenir erros nos modelos de classificação derivados e assegurar uma distribuição equitativa entre as diferentes classes. Este processo foi efetuado manualmente, recorrendo à função sklearn.utils.resample, de forma a extrair uma amostra aleatória de Pokémons para cada tipo, com um tamanho igual ao da classe menos representada. Esta abordagem assegura que cada classe (tipo de Pokémon) tem o mesmo número de

instâncias no *dataset* final, o que promove a imparcialidade dos modelos de classificação e aumenta a robustez das inferências extraídas. A distribuição após o balanceamento está apresentada na **Figura 5**.

```
Distribuição após balanceamento:
type
grass       59
fire        59
water       59
bug         59
normal      59
electric    59
psychic     59
```

Figura 5. Distribuição dos tipos dos Pokemons após balanceamento.

Após o processo de balanceamento do *dataset*, procedeu-se à normalização e preparação textual das descrições dos Pokémons para futura aplicação de modelos de classificação baseados em texto. Para esse efeito, foi definida a função clean_text, responsável por executar uma série de transformações fundamentais para a limpeza linguística e redução da dimensionalidade lexical. Consiste nos seguintes passos:

- Conversão para minúsculas (*lowercasing*): Todo o conteúdo textual é convertido para minúsculas, de modo a eliminar discrepâncias artificiais entre *tokens* que apenas diferem por capitalização, o que reduz a variabilidade do vocabulário e melhora a coerência lexical;
- Remover acentos: Utilizando a biblioteca unidecode, todos os caracteres acentuados são substituídos pelas suas versões não acentuadas, garantindo uniformidade ortográfica e compatibilidade com métodos de tokenização subsequentes;
- Eliminação de palavras irrelevantes e caracteres não alfabéticos: Palavras com valor semântico nulo ou baixo para a tarefa de classificação, tais como “pokemon”, “nan”, “game”, “body” e “one”, são explicitamente removidas, a par de qualquer símbolo ou caractere não alfabético;
- Tokenização e lematização: O texto é segmentado em *tokens*, sobre os quais se aplica a lematização utilizando o WordNetLemmatizer, uma ferramenta da biblioteca NLTK. A lematização consiste na conversão de cada palavra para a sua forma base ou canónica (lemma), reduzindo assim a dispersão lexical e permitindo generalizações mais robustas. Nesta etapa, são também removidas as *stopwords* da língua inglesa

fornecidas pelo NLTK, ou seja, palavras muito frequentes, mas com baixo valor discriminativo (e.g., “the”, “is”, “and”).

A aplicação sistemática da função clean_text à coluna original description deu origem a uma nova coluna, denominada desc_clean, contendo as descrições pré-processadas e otimizadas para análise textual. Esta coluna servirá como base para as fases seguintes de vetorização e treino de modelos de classificação, para garantir que o conteúdo textual foi devidamente normalizado, estruturado e privado de elementos ruidosos.

```
        name                                       description   type  \
0     cacnea  Cacnea is a green, bipedal Pokémon with a roun...  grass
1    dartrix  Dartrix is an avian Pokémon that resembles an ...  grass
2  quilladin  Quilladin is a bipedal, mammalian Pokémon with...  grass
3    servine  Servine is a slim bipedal Pokémon that is prim...  grass
4  abomasnow  Abomasnow is a large, bipedal Pokémon covered ...  grass

                                          desc_clean
0  cacnea green bipedal round cactuslike striatio...
1  dartrix avian resembles owl plumage including ...
2  quilladin bipedal mammalian plantlike feature ...
3  servine slim bipedal primarily green cream und...
4  abomasnow large bipedal covered shaggy white f...
```

Figura 6. Coluna desc_clean.

Prosseguindo para a fase de vetorização do texto, foi implementado o método TF-IDF (*Term Frequency–Inverse Document Frequency*), uma técnica amplamente utilizada em *Text Mining* para a conversão de dados textuais em representações numéricas interpretáveis por modelos de *Machine Learning*. A vetorização foi efetuada através da classe TfidfVectorizer(), com os seguintes parâmetros de filtragem:

- max_df=0.95: Exclui termos que ocorrem em mais de 95% dos documentos, presumindo-se que tais termos têm baixo valor discriminativo por estarem excessivamente difundidos no corpus;
- min_df=3: Ignora termos que aparecem em menos de três documentos, considerando que palavras demasiado raras introduzem ruído e não contribuem significativamente para a generalização do modelo.

A vetorização foi aplicada sobre a coluna desc_clean. O resultado foi armazenado na matriz tfidf_matrix, que constitui a base para as análises subsequentes.

Como parte da análise exploratória do conteúdo semântico do corpus, procedeu-se à aplicação do algoritmo de NMF (*Non-negative Matrix Factorization*), com o objetivo de identificar e interpretar tópicos latentes nos textos. A decomposição NMF permitiu extrair os cinco tópicos mais representativos, cada um constituído por dez termos-chave, facilitando a interpretação qualitativa das temáticas predominantes nas descrições dos Pokémons. No entanto, importa salientar que esta análise teve apenas um caráter exploratório, não tendo os tópicos sido utilizados como *features* nos modelos de classificação.

Para a tarefa de classificação automática do tipo de Pokémon com base nas descrições, optou-se por utilizar exclusivamente a matriz de características TF-IDF, em detrimento de outras abordagens como o CountVectorizer. A escolha do TF-IDF foi motivada por resultados empíricos superiores obtidos durante as iterações experimentais: esta técnica proporciona uma representação mais densa e informativa do conteúdo textual, penalizando termos comuns e realçando os termos mais distintivos de cada documento.

A tfidf_matrix resultante constitui, assim, a representação vetorial final das descrições, sendo utilizada como entrada para os modelos de classificação desenvolvidos nas etapas seguintes.

```
=== Tópicos NMF ===
Tópico 1:  feather | wing | black | beak | avian | marking | head | crest | two | talon
Tópico 2:  fur | flame | ear | sleep | tail | tuft | brown | paw | evolves | nose
Tópico 3:  leaf | shell | green | evolves | head | tree | forest | evolution | mouth | vine
Tópico 4:  fin | water | blue | tail | prey | sea | scale | pectoral | dorsal | light
Tópico 5:  power | form | electricity | yellow | blue | head | two | capable | known | white
```

Figura 7. Tópicos NMF aplicados.

Em continuidade da análise exploratória dos dados, procedeu-se à soma dos valores TF-IDF por termo, permitindo assim identificar e ordenar os dez termos mais relevantes com base nesta métrica, conforme a Figura 8:

```
Top 10 termos pelo total de TF-IDF:
black: 15.01
head: 14.97
yellow: 14.34
two: 13.57
tail: 13.20
evolves: 12.34
fur: 12.23
eye: 11.42
red: 11.30
blue: 11.27
```

Figura 8. Os dez termos mais relevantes obtidos pela métrica

3.4 Modelação dos dados

Para a fase de modelação preditiva, foram utilizados três algoritmos de aprendizagem supervisionada: Random Forest, Regressão Logística e Naive Bayes, aplicados com o objetivo de classificar os Pokémons com base nas suas descrições textuais.

Antes da aplicação dos modelos, definiram-se as seguintes componentes do problema de classificação:

- *Features*: Representadas pela matriz tfidf_matrix, obtida através da vetorização das descrições textuais com TF-IDF.
- *Target*: A variável de saída *type*, correspondente ao tipo de cada Pokémon.

O *dataset* foi dividido em conjuntos de treino e teste com uma proporção de 80% para treino e 20% para teste, utilizando a função train_test_split() da biblioteca scikit-learn, para assegurar uma divisão estratificada para manter a representatividade das classes. A avaliação do desempenho de cada modelo foi realizada através das métricas accuracy_score e classification_report, aplicadas sobre o conjunto de teste.

2.2.1. *Random Forest*

O Random Forest é um algoritmo baseado em conjuntos de árvores de decisão (*ensemble learning*), que combina múltiplas árvores construídas sobre subconjuntos

aleatórios dos dados e das variáveis. Esta abordagem permite aumentar a precisão preditiva e reduzir o risco de overfitting, ao agregar os resultados de várias árvores independentes por votação. O modelo é especialmente eficaz em problemas de classificação com múltiplas classes e dados de alta dimensionalidade.

2.2.2. *Regressão Logística*

A Regressão Logística é um modelo linear de classificação, que estima a probabilidade de pertença de uma instância a uma determinada classe, com base numa função logística (sigmoide ou softmax, consoante o número de classes). É amplamente utilizada tanto em classificação binária como multiclasse, sendo valorizada pela sua interpretabilidade e robustez em contextos onde as classes são aproximadamente separáveis de forma linear. No contexto atual, serviu como uma abordagem de base sólida para comparação com modelos mais complexos.

2.2.3. *Naive Bayes*

O Naive Bayes é um classificador probabilístico que se baseia no Teorema de Bayes, assumindo uma independência condicional entre as variáveis preditoras. Esta suposição, apesar de simplificadora, torna o modelo computacionalmente eficiente e extremamente adequado para tarefas de classificação de texto, como é o caso presente, em que os dados foram representados por uma matriz esparsa TF-IDF. O modelo destaca-se pela sua rapidez de treino e bons resultados em ambientes de elevada dimensionalidade, mesmo com um volume moderado de dados.

3.5 Avaliação do Modelo

Para a avaliação do desempenho dos modelos de classificação, foi adotado o método de Holdout, com a divisão dos dados previamente efetuada na proporção de 80% para treino e 20% para teste. Esta abordagem permitiu uma análise comparativa direta entre os três algoritmos selecionados: Logistic Regression, Random Forest e Naive Bayes. A avaliação foi feita com base em métricas de desempenho padrão, amplamente reconhecidas na literatura de *machine learning*, nomeadamente:

- *Accuracy*; Proporção de previsões corretas face ao total de previsões realizadas. Esta métrica fornece uma visão global da performance do modelo, embora possa ser enviesada em situações de classes que não estejam propriamente balanceadas.
- *Precision*: Mede, entre todas as instâncias que o modelo previu como pertencentes a uma determinada classe, quantas eram efetivamente dessa classe. Esta métrica é especialmente importante quando os falsos positivos têm um custo elevado.
- *Recall*: Mede, entre todas as instâncias que realmente pertencem a uma determinada classe, quantas foram corretamente identificadas pelo modelo. Esta métrica torna-se crítica em contextos onde a detecção de todas as instâncias relevantes é prioritária.
- *F1-score:* Corresponde à média harmónica entre a *precision* e o *recall*, o que oference uma métrica equilibrada que é particularmente útil quando existe assimetria entre estas duas medidas. O F1-score penaliza fortemente valores extremos, o que promove um compromisso equilibrado entre identificação e precisão.

Tabela 1. Valores obtidos para as métricas em relação aos modelos utilizados.

Modelo	Accuracy	Precision (avg)	Recall (avg)	F1-Score (avg)
Logistic Regression	0.82	0.86	0.82	0.83
Random Forest	0.76	0.79	0.76	0.77
Naive Bayes	0.81	0.83	0.81	0.81

Entre os três modelos avaliados, a Regressão Logística destacou-se como o modelo com melhor desempenho global, onde alcançou uma *accuracy* de 82% e apresenta valores superiores nas restantes métricas (*precision*, *recall* e *F1-score*) em comparação com os demais algoritmos. Este modelo revelou-se particularmente eficaz na classificação dos tipos *bug*, *electric*, *fire* e *grass*, o que demonstra não só elevada capacidade discriminativa, como também consistência nas previsões entre diferentes classes.

O modelo Naive Bayes, apesar da sua simplicidade e das suposições fortes de independência entre atributos, excedeu as expectativas, obtendo uma *accuracy* de 81%. Apresentou um

desempenho competitivo, com resultados positivos nas classes *electric*, *grass* e *normal*. A sua eficiência reforça a adequação deste modelo a tarefas de classificação textual de alta dimensionalidade, como aquelas baseadas em representações TF-IDF.

Em contraste, o Random Forest revelou um desempenho inferior relativamente aos outros dois modelos. Para além de apresentar uma *accuracy* mais baixa, observou-se uma maior variabilidade nos valores de *recall* e *F1-score* entre as diferentes classes, o que pode indicar instabilidade na capacidade do modelo para generalizar de forma equilibrada. Esta dispersão sugere que, apesar da sua robustez teórica, o Random Forest pode não ser ideal em contextos com dados textuais vetorizados de forma esparsa, como é o caso presente.

Com base nos resultados obtidos, conclui-se que a Regressão Logística é o modelo mais adequado para este conjunto de dados, não só pelo seu desempenho médio superior, mas também pela robustez e equilíbrio demonstrados na classificação das várias classes. Este desempenho confirma a sua adequação a problemas de classificação multiclasse com características textuais bem representadas em espaços vetoriais densos como os originados pelo TF-IDF.

4 Conclusões

De forma geral, o projeto atingiu com sucesso o seu principal objetivo: o desenvolvimento de duas ferramentas distintas e complementares, sendo esta um sistema de pesquisa de Pokémons com base em linguagem natural (Pokémon Searcher) e um modelo de predição automática do tipo de Pokémon a partir da descrição textual (*Type Predictor*).

O Pokémon Searcher demonstrou elevada flexibilidade, pelo que permitiu a integração de técnicas de correspondência aproximada (*fuzzy matching*) com filtragem baseada em atributos descritivos (como tipo, altura, peso ou ataque). Esta combinação permitiu obter resultados semanticamente coerentes e alinhados com as intenções expressas pelo utilizador nas consultas textuais, mesmo na presença de variações linguísticas ou erros ortográficos.

Relativamente ao Type Predictor, a utilização da vetorização TF-IDF em conjunto com a análise exploratória de tópicos via NMF, permitiu explorar padrões semânticos subjacentes nas descrições dos Pokémons. Após a aplicação de três algoritmos de classificação, Random Forest, Regressão Logística e Naive Bayes, concluiu-se que a Regressão Logística apresentou o melhor desempenho global. O Naive Bayes, apesar da sua simplicidade, revelou um desempenho competitivo, ficando apenas ligeiramente atrás. Estes resultados demonstram a eficiência da vetorização TF-IDF, mesmo

em contextos contemporâneos onde métodos mais complexos são frequentemente utilizados, e reforçam a importância crítica de um pré-processamento textual bem estruturado.

Apesar dos resultados satisfatórios, sentimos que o projeto apresentou algumas limitações relevantes tal como a facto de que não foi possível integrar nem testar modelos de linguagem mais avançados, como *Word2Vec*, *GloVe* ou *transformers* pré-treinados, que poderiam contribuir potencialmente para uma melhoria significativa do desempenho dos classificadores e da relevância semântica dos resultados do motor de busca.

De um modo geral, apesar de diversas dificuldades iniciais, sentimos um grande sentimento de satisfação ao realizar este trabalho, tanto por ser esta a área com que mais nos identificamos e pretendemos nos especializar no nosso próximo ciclo de estudos, como pela possibilidade livre da escolha do tema, que nos permitiu estabelecer uma ponte entre os nossos gostos pessoais e a Unidade Curricular.

5 Referências

Breiman, L. (2001). Random forests. Machine learning, 45(1), 5–32.

Camacho-Collados, J., & Pilehvar, M. T. (2018). From Word to Sense Embeddings: A Survey on Vector Representations of Meaning. Journal of Artificial Intelligence Research, 63, 743–788.

Jurafsky, D., & Martin, J. H. (2021). Speech and Language Processing (3rd ed. draft). Stanford University.

Kowsari, K., Meimandi, K. J., Heidarysafa, M., Mendu, S., Barnes, L. E., & Brown, D. E. (2019). Text Classification Algorithms: A Survey. Information, 10(4), 150.

Manning, C. D., Raghavan, P., & Schütze, H. (2008). Introduction to Information Retrieval. Cambridge University Press.

Minaee, S., Kalchbrenner, N., Cambria, E., Nikzad, N., Chenaghlu, M., & Gao, J. (2021). Deep learning-based text classification: A comprehensive review. ACM Computing Surveys (CSUR), 54(3), 1–40.

Zhang, T., Jin, R., & Zhou, Z.-H. (2010). Understanding bag-of-words model: a statistical framework. International Journal of Machine Learning and Cybernetics, 1(1), 43–52.

Amos, D. (2024, December 21). *A practical introduction to web scraping in Python*. https://realpython.com/python-web-scraping-practical-introduction/

GeeksforGeeks. (2024, September 29). *FuzzyWuzzy Python Library*. GeeksforGeeks. https://www.geeksforgeeks.org/fuzzywuzzy-python-library/

Bulbapedia. (2024, October 30). *Bulbapedia, the community-driven Pokémon encyclopedia*. Bulbapedia. https://bulbapedia.bulbagarden.net/wiki/Main_Page

TFIDFVectorizer. (n.d.). Scikit-learn. https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html

Natural language processing fundamentals: tokens, N-Grams, and Bag-of-Words models. (2023, November 7). https://zilliz.com/learn/introduction-to-natural-language-processing-tokens-ngrams-bag-of-words-models

The complete Pokemon Dataset. (2017, September 29). Kaggle. https://www.kaggle.com/datasets/rounakbanik/pokemon